

CCReimagined

The CodeComplete data-class generator, rebuilt for macOS, Windows and Linux — and no longer tied to SQL Server.

Point it at a table on any of four database engines and it writes the same data-abstraction class you have been using for years. The shape of that class never changes. What changes is everything underneath it: the client library, the parameter types, the SQL quoting, and the way a new row's key comes back.

This guide shows the tool running against live PostgreSQL and SQL Server instances, and puts the generated output for the same table side by side across all four engines so you can see exactly what stays the same and what adapts.

Engines	Runs on	Replaces	Repository
SQL Server · PostgreSQL MySQL / MariaDB · SQLite	macOS, Windows, Linux Avalonia, .NET 10	TAICodeComplete (WinForms, SQL Server only)	Harlock123/CCReimagined private

What it does, and what has changed

The job is the same as it always was. You choose a table or view; the tool reads its columns from the database catalog and writes a partial C# class with a property per column and a full set of CRUD methods. You still choose which column is the key, and which columns become parameters on the list getters.

Three things are different:

- **It runs natively on macOS, Windows and Linux.** The WinForms original ran on Windows only.
- **It speaks four engines,** and the code it writes targets whichever one you point it at.
- **The engine you browse and the engine you emit for are separate choices.** Read a SQL Server schema, emit an Npgsql class. That is what porting a table between engines needs.

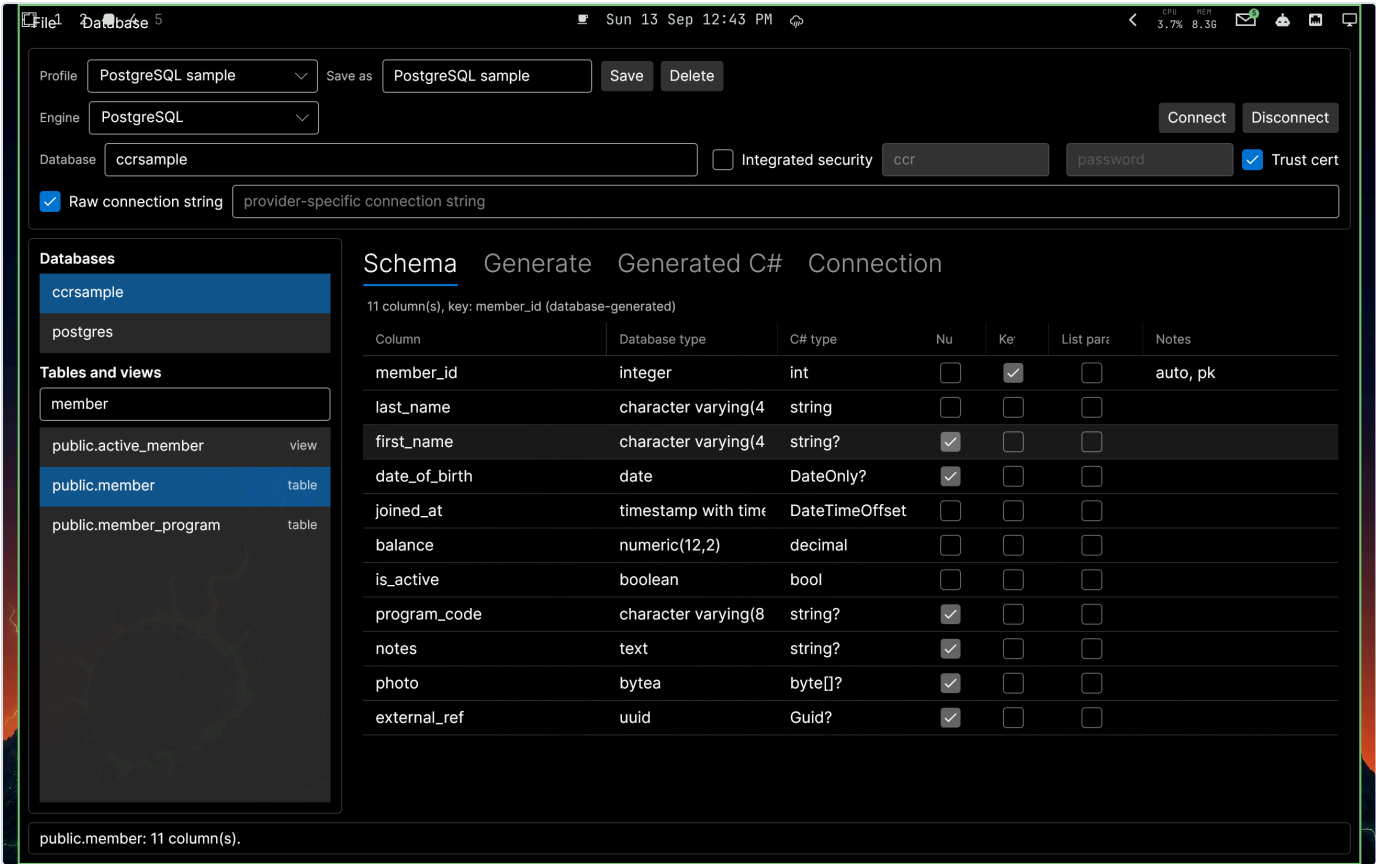
Five minutes to your first class

- 1 **Pick the engine** and fill in the server, database and credentials. Press **Connect**.

For PostgreSQL, untick *Integrated security* first — it is ticked by default because peer authentication is its equivalent, and while it is ticked no password is sent.

- 2 **Press Save** with a name. The connection comes back next launch. The password is never written to disk, so that one field is retyped.
- 3 **Pick a table or view** from the list. Type in the filter box to narrow it.
- 4 **Check the key column** in the grid. The tool picks one and tells you how confident it is; the tick is yours to move.
- 5 **Tick any columns** you want a GetListBy... method for.
- 6 **Generate class.** The output tab comes forward. **Copy**, or **Save as...** into your project.

Connecting



Connected to a live PostgreSQL 18 instance. The left rail lists databases, then tables and views. The grid is the discovered schema for `public.member` — and the column that matters most here is **C# type**, which is covered on the next page.

The generated code is database-type aware

Every provider maps its own native type names onto one neutral set of CLR types during discovery. That mapping is what you see in the **C# type** column, and it is what the generated properties are declared as.

C# PROPERTY	SQL SERVER	POSTGRESQL	MYSQL	SQLITE
int / long	int, bigint	integer, bigint	INT, BIGINT	INTEGER
string	nvarchar, varchar, text	varchar, text, json	VARCHAR, TEXT, JSON	VARCHAR, TEXT, CLOB
decimal	decimal, money	numeric, money	DECIMAL	DECIMAL
bool	bit	boolean	TINYINT(1)	BOOLEAN
DateOnly	date	date	DATE	DATE
DateTime	datetime2	timestamp	DATETIME	DATETIME
DateTimeOffset	datetimeoffset	timestampz	—	—
Guid	uniqueidentifier	uuid	CHAR(36)	GUID
byte[]	varbinary, image	bytea	BLOB	BLOB

Nullability is carried through, not guessed

A nullable column becomes a nullable property — `string?`, `DateOnly?`, `decimal?`. The classes the old tool wrote used sentinel values instead ("", a minimum date), so "not set" and "legitimately empty" were indistinguishable once the row came back. They are distinguishable now.

The same column, four engines

`date_of_birth` is declared `date` on PostgreSQL and SQL Server, `DATE` on MySQL, and `DATE` on SQLite. In all four the property is identical, and only the parameter type differs:

SQL SERVER · MICROSOFT.DATA.SQLCLIENT

```
public DateOnly? date_of_birth { get; set; }

Bind(command, "@p_date_of_birth",
    System.Data.SqlDbType.Date,
    _date_of_birth?.ToDateTime(TimeOnly.MinValue));
```

POSTGRESQL · NPGSQL

```
public DateOnly? date_of_birth { get; set; }

Bind(command, "@p_date_of_birth",
    NpgsqlTypes.NpgsqlDbType.Date,
    _date_of_birth?.ToDateTime(TimeOnly.MinValue));
```

MYSQL · MYSQLCONNECTOR

```
public DateOnly? date_of_birth { get; set; }

Bind(command, "@p_date_of_birth",
    MySqlConnection.MySqlDbType.Date,
    _date_of_birth?.ToDateTime(TimeOnly.MinValue));
```

SQLITE · MICROSOFT.DATA.SQLITE

```
public DateOnly? date_of_birth { get; set; }

Bind(command, "@p_date_of_birth",
    Microsoft.Data.Sqlite.SqliteType.Text,
    _date_of_birth?.ToDateTime(TimeOnly.MinValue));
```

SQLite stores dates as text — that is the engine's design, not a shortcut. The property is still `DateOnly?`, so your code does not care.

The patterns are identical across every engine

This is the part that matters for a team. Whatever database a class was generated against, it exposes exactly the same members, with the same names and the same signatures. Nobody has to learn a second idiom because a project moved to Postgres.

MEMBER	WHAT IT DOES
<code>Initialize()</code>	Resets every mapped column to its default.
<code>CopyFields(DbDataReader)</code>	Copies the current row in. Columns absent from the result set are left alone, so a partial SELECT is fine.
<code>ReadAsync(id)</code>	Loads one row by key. Returns <code>false</code> when there is no such row.
<code>AddAsync()</code>	Inserts, and reads the database-generated key back into the property.
<code>UpdateAsync()</code>	Writes every writable column back. Returns rows affected.
<code>DeleteAsync()</code>	Deletes by key. Returns rows affected.
<code>RecExistsAsync(id)</code>	True when a row with that key exists.
<code>ReadAsDataTableAsync(id)</code>	The portable replacement for the old <code>ReadAsDataSet</code> .
<code>GetAllAsync(cs)</code>	Every row, optionally row-capped.
<code>GetListByColumnAsync(cs, value)</code>	One per column you ticked as a list parameter.

Each async method has an optional blocking wrapper (`Read`, `Add`, `Update`, `Delete`...) for callers that cannot await.

What adapts underneath

Four things change per engine, and nothing else:

	SQL SERVER	POSTGRESQL	MYSQL	SQLITE
Client	SqlConnection	NpgsqlConnection	MySqlConnection	SqliteConnection
Parameter type	SqlDbType	NpgsqlDbType	MySqlDbType	SqliteType
Quoting	[name]	"name"	`name`	"name"
New key	SCOPE_IDENTITY()	RETURNING	LAST_INSERT_ID()	last_insert_rowid()

You can see all four in the INSERT statement the generator emits for the same table:

SQL SERVER

```
INSERT INTO [dbo].[member] (  
    [last_name], [first_name], [date_of_birth],  
    ""  
)  
VALUES (  
    @p_last_name, @p_first_name,  
    @p_date_of_birth, ...  
);  
SELECT CAST(SCOPE_IDENTITY() AS bigint);
```

POSTGRESQL

```
INSERT INTO "public"."member" (  
    "last_name", "first_name", "date_of_birth",  
    ""  
)  
VALUES (  
    @p_last_name, @p_first_name,  
    @p_date_of_birth, ...  
)  
RETURNING "member_id"
```

MYSQL / MARIADB

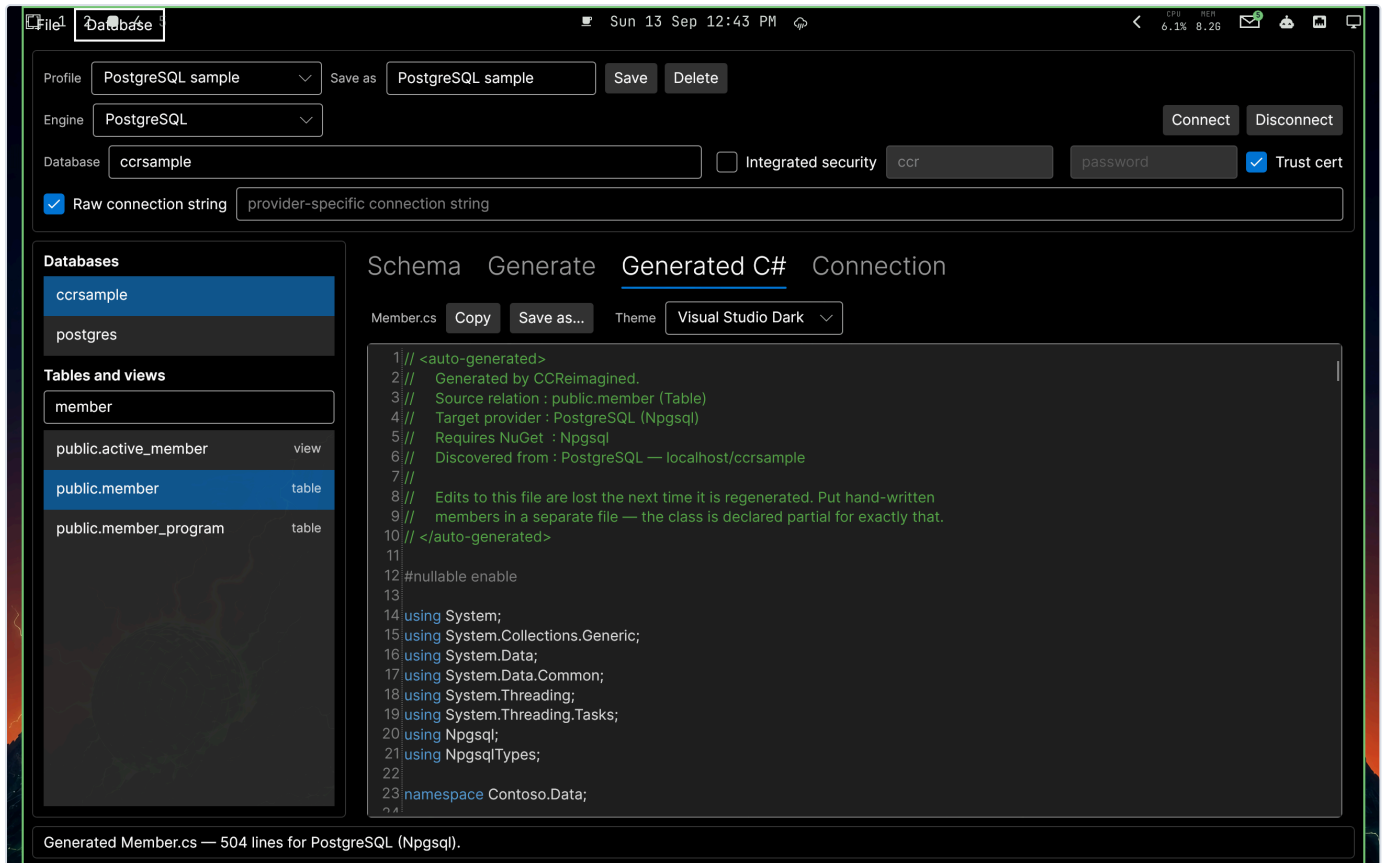
```
INSERT INTO `member` (  
    `last_name`, `first_name`, `date_of_birth`,  
    ""  
)  
VALUES (  
    @p_last_name, @p_first_name,  
    @p_date_of_birth, ...  
);  
SELECT LAST_INSERT_ID();
```

SQLITE

```
INSERT INTO "member" (  
    "last_name", "first_name", "date_of_birth",  
    ""  
)  
VALUES (  
    @p_last_name, @p_first_name,  
    @p_date_of_birth, ...  
);  
SELECT last_insert_rowid();
```

The SQL lives in const raw string literals at the top of the generated class, so it is reviewable in a diff rather than assembled at runtime.

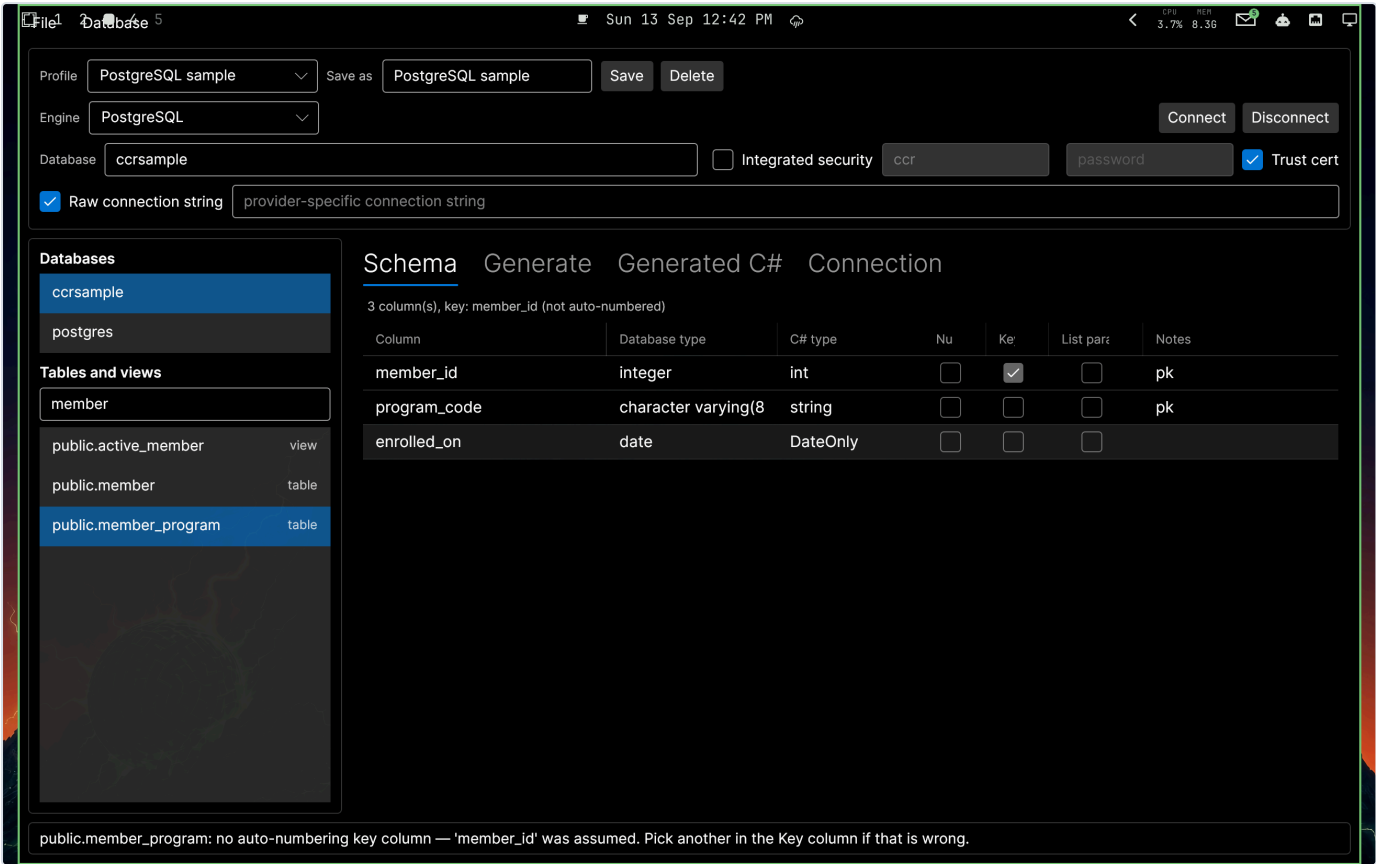
The output



The same table, generated for PostgreSQL: 504 lines, using `Npgsql`. The header records the source relation, the target provider and the NuGet package the file needs, so a class that turns up in a code review explains itself.

The class is `partial` and the header says so: regenerating overwrites the file, so hand-written members belong in a second file beside it.

Tables the old tool refused



A composite-key table. There is no auto-numbering column, so the tool names the one it assumed and carries on — the tick in the **Key** column is yours to move. The original stopped at a modal dialog here.

Worth knowing before you regenerate an old class

Some behaviours were corrected rather than carried across. If you are replacing a class the old tool wrote, expect these:

THEN	NOW
Nullable columns used sentinels	Nullable columns are nullable properties. Empty-string-as-NULL is still available as an opt-in.
INTERVIEWER renamed because it contains int	Reserved words are matched exactly. Only a column actually named int or class gets escaped.
Columns ordered alphabetically	Columns follow the table's declared order.
Only IDENTITY counted as generated	Also serial, AUTO_INCREMENT, and SQLite's rowid alias.
ReadAsDataSet	ReadAsDataTableAsync — not every provider ships a DataAdapter.
Synchronous only	Async throughout, with blocking wrappers kept.

How far it is verified

For each engine the test suite creates a table, discovers it, generates the class, **compiles that class with Roslyn**, and then drives insert, key read-back, read, update, exists, both list getters and delete **against a real server**. Generated code that does not build, or does not round-trip a row, fails the build.

Getting the sample databases

dev/sample-databases in the repository has a Docker Compose stack for all four engines and a schema built to exercise the awkward cases — keyless tables, composite keys, computed columns, and column names that are C# keywords. Its README lists every connection parameter.

Running it yourself

The repository is Harlock123/CCReimagined. You need the .NET 10 SDK and nothing else.

```
# build and run the app
git clone https://github.com/Harlock123/CCReimagined.git
cd CCReimagined
dotnet run --project src/CCReimagined.App

# optional: sample databases to try it against
cd dev/sample-databases
docker compose up -d                # PostgreSQL, MySQL, MariaDB
./make-sqlite.sh                    # SQLite, no container needed
docker compose --profile mssql up -d # SQL Server (amd64 hosts)
./seed-sqlserver.sh                 # apply the sample schema to it
```

SHORTCUT	DOES
Ctrl+S / Cmd+S	Save the generated class to a file
Ctrl+Q / Cmd+Q	Quit — also File › Exit
Database menu	Connect, Disconnect, Generate class

Connection profiles are stored per user — ~/.config/CCReimagined on Linux, ~/Library/Application Support/CCReimagined on macOS, %APPDATA%\CCReimagined on Windows. **Passwords are never written to disk**, including any password inside a saved raw connection string, so the file is safe to sync between machines.